

Imam Abdulrahman Bin Faisal University
College of Computer Science & Information Technology
Department of Computer Engineering

Algorithm Analysis and Design
Term 2 – 2025-2026

Project Report on
Shortest Path Algorithms Comparison for GPS Navigation

Supervised by **Dr. Nasro Min Allah**

Submission Date : 21 / 05 / 2026

Group 4		
Student Name	Student ID	Role
Omar Fawzi Hawsawi	2240007072	Leader
Fadel Abbas Al-Asaad	2240002437	Member
Mohammed Nasser Almarhoon	2240005893	Member
Nawaf Khaled Alwabel	2240001111	Member
Mohammed Othman Alkodir	2260040121	Member
Abdulaziz Khaled Alhassan	2240006848	Member

Table of Contents

Introduction	5
Section I: Theoretical	5
1. Definition of the Problem	5
2. Pseudocode for the Selected Algorithms	6
3. How the Chosen Algorithms Solve the Problem	7
4. Literature Review of the Chosen Algorithms	8
5. Comparison of Chosen Algorithms Theoretically in Terms of Time and Space	9
Complexity	
Section II: Experimental Results	11
1. Machine specifications	11
2. The timing mechanism	12
3. Sample size.....	12
4. What time was reported?	12
5. How did you select the inputs?.....	12
6. Did you use the same inputs for all algorithms?	12
Experimental results:.....	13
1. Dijkstra's algorithm	13
2. Bellman-Ford algorithm.....	17
3. Floyd-Warshall algorithm.....	19
4. A* algorithm	21
Find the computational complexity of each algorithm (analyzing line of codes).....	25
Find $T(n)$ and order of growth concerning the dataset size	27
Conclusion	31
Appendix.....	32

List of Tables

Table 1: Machine specifications.....	11
Table 2: Dijkstra's algorithm execution time for small inputs.....	13
Table 3: Dijkstra's algorithm experimental vs theoretical comparison	14
Table 4: Dijkstra's algorithm execution time for large inputs.....	15
Table 5: Dijkstra's algorithm - Large input experimental vs theoretical comparison	16
Table 6: Bellman-Ford algorithm execution time for small inputs	17
Table 7: Bellman-Ford algorithm - Small input experimental vs theoretical comparison.....	18
Table 8: Floyd-Warshall algorithm execution time for small inputs	19
Table 9: A algorithm execution time for small inputs*.....	21
Table 10: A algorithm execution time for large inputs*.....	23
Table 11: Algorithm comparison for small input.....	24
Table 12: Algorithm comparison for large input	24
Table 13: Final comparison table (4 algorithms).....	25

List of Figures

Figure 1: Dijkstra execution time vs input size (small inputs)	14
Figure 2: Dijkstra execution time vs input size (large inputs).....	16
Figure 3: Bellman-Ford execution time vs input size.....	18
Figure 4: Floyd-Warshall execution time vs input size.....	20
Figure 5: A* execution time vs input size (small inputs)	22
Figure 6: A* execution time vs input size (large inputs)	24
Figure 7: Best case graph using small input	28
Figure 8: Best case graph using large input.....	28
Figure 9: Worst case graph using small input.....	29
Figure 10: Worst case graph using large input.....	29
Figure 11: Space complexity comparison with small inputs.....	30
Figure 12: Space complexity comparison with large inputs.....	30

1. Introduction

In modern urban environments, the ability to find the shortest path between two locations is essential for a wide range of applications, including GPS navigation systems (such as Google Maps and Waze), ride-sharing services (Uber, Careem), logistics and delivery planning (Amazon, FedEx), and network routing protocols.

This project aims to test and compare **four shortest path algorithms**:

1. **Dijkstra's Algorithm**
2. **Bellman-Ford Algorithm**
3. **Floyd-Warshall Algorithm**
4. **A (A-Star) Algorithm***

The selected algorithms are evaluated based on their execution time, accuracy, and ability to handle different graph sizes. The goal is to determine which algorithm is the most efficient for finding the shortest path in a weighted graph representing 500 cities with distances between them.

Section I: Theoretical

1. Definition of the Problem

The problem we are dealing with here is figuring out the shortest driving distance between two cities out of a total of 500 cities that are all connected with distances. Think of it like a GPS trying to tell you the fastest way to get from point A to point B. The system we built needs to take a starting city and a destination, then calculate the best possible route based on the distances between cities. After that, it should show both the full path and the total distance. We also had to measure how long each algorithm takes to finish so we could compare their performance. One important thing is that all the distances are non-negative, which makes sense for real-world maps - you cannot have a negative distance between two places.

This kind of problem shows up everywhere in real life. GPS apps like Google Maps or Waze use it all the time. Ride-sharing services like Uber and Careem rely on it too. Delivery companies like Amazon or FedEx need it for planning their routes. Even emergency services like ambulances and fire trucks depend on finding the quickest path. Network routing protocols also work similarly when they are trying to find the best way to send data across the internet.

2. Pseudocode for the Selected Algorithms

2.1 Dijkstra's Algorithm

Dijkstra(Graph, source):

1. Initialize set S to track finalized vertices
2. Set $\text{distance}[\text{source}] = 0$, all others = infinity
3. Push $(0, \text{source})$ into priority queue
4. While S does not contain all vertices:
 - a. Pick u (not in S) with smallest distance
 - b. Add u to S
 - c. For each neighbor v of u :
 - i. If v not in S and $\text{distance}[u] + \text{weight}(u,v) < \text{distance}[v]$:
Update $\text{distance}[v]$
5. Return distance array

2.2 Bellman-Ford Algorithm

BellmanFord(Graph, source):

1. Initialize $\text{distance}[\text{source}] = 0$ and $\text{distance}[\text{all other vertices}] = \infty$
2. For $i = 1$ to $|V|-1$:
 - a. For each edge (u, v) with weight w :
 - i. If $\text{distance}[u] + w < \text{distance}[v]$:
 $\text{distance}[v] = \text{distance}[u] + w$
3. For each edge (u, v) with weight w :
 - a. If $\text{distance}[u] + w < \text{distance}[v]$:
Return "Negative cycle detected"
4. Return distance array

2.3 Floyd-Warshall Algorithm

FloydWarshall(Graph):

1. Initialize $\text{dist}[u][v] = \text{weight}(u,v)$ for direct edges, ∞ otherwise
2. Initialize $\text{dist}[u][u] = 0$ for all u
3. For $k = 1$ to $|V|$:
 - a. For $i = 1$ to $|V|$:
 - b. For $j = 1$ to $|V|$:
 - i. If $\text{dist}[i][k] + \text{dist}[k][j] < \text{dist}[i][j]$:
 $\text{dist}[i][j] = \text{dist}[i][k] + \text{dist}[k][j]$
4. Return dist matrix

2.4 A* (A-Star) Algorithm

AStar(Graph, start, goal):

1. Initialize open_set with start node and closed_set as empty
2. For each node, initialize g_score = infinity, f_score = infinity
3. Set g_score[start] = 0 and f_score[start] = heuristic(start, goal)
4. While open_set is not empty:
 - a. Let current = node in open_set with lowest f_score
 - b. If current = goal:
Return the path from start to goal
 - c. Remove current from open_set and add to closed_set
 - d. For each neighbor of current:
 - i. If neighbor is in closed_set, continue
 - ii. Compute tentative_g = g_score[current] + weight(current, neighbor)
 - iii. If tentative_g < g_score[neighbor]:
Update neighbor's g_score and f_score
Add neighbor to open_set if not already present
5. If goal is not reachable, return failure

3. How the Chosen Algorithms Solve the Problem

Each of the four algorithms we picked – Dijkstra, Bellman-Ford, Floyd-Warshall, and A* – has its own way of finding the shortest path between cities.

Let's start with **Dijkstra**. This one works by gradually exploring outward from the starting point. It always picks the closest city that hasn't been processed yet and checks its neighbors. By using a priority queue, it makes sure it's always looking at the most promising option next. It's really reliable as long as none of the distances are negative, which is perfect for GPS navigation.

Now **Bellman-Ford** is more powerful in one way – it can actually handle negative weights if they exist. But that extra power comes with a cost: it's much slower. It works by going through all the connections repeatedly, relaxing them over and over. For our project, though, negative distances don't really make sense between cities, so this algorithm isn't the best fit.

Floyd-Warshall takes a completely different approach. Instead of finding just one route, it figures out the shortest path between every possible pair of cities all at once using dynamic programming. That sounds useful, but the problem is it's extremely slow when you have a lot of cities. For 500 cities, it needs about 125 million operations, which is way too much for real-time use.

Finally, **A*** is like an improved version of Dijkstra. The main difference is that it uses something called a heuristic – basically an educated guess about how far the destination might be. This helps it focus on paths that seem more promising. But here's the catch: if you don't have good heuristic data (like city coordinates), it ends up working almost exactly like Dijkstra anyway.

4. Literature Review of the Chosen Algorithms

- **Dijkstra's Algorithm** - There is a paper from IEEE called "Comparison Study of Three Shortest Path Algorithms" that talks about how Dijkstra's algorithm is used to find shortest paths in graphs. The paper points out that Dijkstra does a great job when all the weights are non-negative, especially for things like traffic routing and network design. But the authors also mention that if the graph is really large and keeps changing, Dijkstra can start to struggle because it basically has to check everything.
- **Bellman-Ford Algorithm** - Another research paper titled "The Bellman-Ford Algorithm for Shortest Paths with Negative Weights" explains how this algorithm can do something Dijkstra cannot: handle negative weights and detect negative cycles. The downside is that it runs in $O(n^3)$ time, which makes it much slower for sparse graphs. Still, it is the go-to choice for financial networks where you might have arbitrage situations.
- **Floyd-Warshall Algorithm** - This algorithm was published by Robert Floyd back in 1962, and then independently by Stephen Warshall. There is a paper called "Algorithm 97: Shortest Path" that presents it. The algorithm is very elegant and simple to understand because it uses dynamic programming. But the $O(n^3)$ time complexity is a real problem. You would not want to use this for anything real-time, but it is fine for offline planning where you need all the routes in advance.
- **A* Algorithm*** - The classic paper here is "A Formal Basis for the Heuristic Determination of Minimum Cost Paths" by Hart, Nilsson, and Raphael from 1968. This is where A* was first introduced. The paper shows that A* can be much faster than Dijkstra in situations where you have a good heuristic because it does not waste time exploring irrelevant paths. That is why it is so popular in robotics and video games where the environment is dynamic and you need quick adjustments.

5. Comparison of Chosen Algorithms Theoretically in Terms of Time and Space Complexity

5.1 Dijkstra's Algorithm

Dijkstra's algorithm operates by iteratively selecting the vertex with the smallest distance from the source and updating the distances of its neighbors. It uses a priority queue for efficient distance selection.

Best Case:

The best-case scenario occurs when the graph is sparse and the source is directly connected to the target node. In this case, the time complexity is **$O(n \log n)$** , where n is the number of cities.

Average Case:

On average, the algorithm processes each edge multiple times, resulting in a time complexity of **$O(n^2)$** .

Worst Case:

The worst-case scenario occurs when the graph is dense (many edges), resulting in **$O(n^2)$** .

Space Complexity:

The algorithm requires space to store the priority queue and distance array, yielding a space complexity of **$O(n)$** .

5.2 Bellman-Ford Algorithm

Bellman-Ford takes a different approach by relaxing all edges multiple times. This makes it more flexible but slower.

Best Case:

The best-case scenario results in **$O(n^2)$** .

Average Case:

On average, the algorithm performs all $|V|-1$ relaxations, resulting in **$O(n^3)$** .

Worst Case:

The worst case also requires **$O(n^3)$** operations, making it significantly slower than Dijkstra.

Space Complexity:

Bellman-Ford requires **$O(n)$** space to store distances and predecessors.

5.3 Floyd-Warshall Algorithm

Floyd-Warshall solves the all-pairs shortest path problem using dynamic programming.

Best/Average/Worst Case:

All cases run in $O(n^3)$ because the algorithm always iterates through all triples (k, i, j) .

Space Complexity:

The algorithm requires $O(n^2)$ space to store the distance matrix.

5.4 A* Algorithm

A* enhances Dijkstra by using a heuristic to guide the search toward the goal.

Best Case:

When the heuristic perfectly predicts the shortest path, A* explores only the optimal route. The complexity is $O(n)$.

Average Case:

A* typically performs better than Dijkstra as it explores fewer nodes, averaging $O(n \log n)$.

Worst Case:

In the absence of an effective heuristic, A* degenerates to Dijkstra's performance, yielding $O(n^2)$.

Space Complexity:

A* requires $O(n)$ space for the open and closed sets.

Aspect	Dijkstra	Bellman-Ford	Floyd-Warshall	A*
Main Objective	Find shortest path from source to all nodes in weighted graph	Find shortest path handling negative weights	Find shortest paths between allpairs of nodes	Find shortest path from start to goal using heuristic
Decision Made Based On	Node with smallest tentative distance	Relaxing all edges repeatedly	Dynamic programming across all triples (k,i,j)	Combining path cost $g(n)$ + heuristic $h(n)$
Basic Concept	Iterative exploration using priority queue	Repeated edge relaxation	All-pairs dynamic programming	Informed search with heuristic guidance
Approach Used	Priority queue (greedy)	Iterative relaxation (dynamic)	Triple nested loops	Priority queue + heuristic

Aspect	Dijkstra	Bellman-Ford	Floyd-Warshall	A*
Often Used In	GPS, Maps, Network routing	Financial networks, Graphs with negative weights	Network infrastructure, Small graphs	Games, Robotics, AI pathfinding
Disadvantages	Cannot handle negative weights	Very slow $O(V \times E)$	Extremely slow $O(V^3)$, large memory	Performance depends on heuristic quality

Section II: Experimental Results

1. Machine Specifications

Component	Specification
Processor	ARM
RAM	8.0 GB
Storage	245 GB
Operating System	Darwin 25.0.0
Programming Language	Python 3.13.7
Libraries Used	pandas, heapq, time, math

Table 1: Machine specifications

2. The Timing Mechanism

We used **time.perf_counter()** for high-resolution timing. Each reported time is the average of 20 runs, converted to microseconds.

```
start_time = time.perf_counter()
result = algorithm(graph, source, target)
end_time = time.perf_counter()
execution_time = (end_time - start_time) * 1_000_000 #microseconds
```

3. Sample Size

Our dataset had 500 cities labeled City1 through City500. For testing, we decided to look at smaller groups and larger groups separately. The small ones had 100, 150, 200, and 250 cities. The large ones had 300, 350, 400, 450, and 500 cities. To make sure our results weren't just random noise, we ran each test 20 times for each graph size and took the average.

4. What Time Was Reported?

The times reported are the **average execution time** of 20 runs for each algorithm and each input size. All times are reported in **microseconds (μs)** unless otherwise specified.

5. How Did You Select the Inputs?

We defined:

- **4 small graphs** with 100, 150, 200, and 250 nodes respectively
- **5 large graphs** with 300, 350, 400, 450, and 500 nodes respectively

For each test, we selected:

- **Source city:** The first city in the subgraph (e.g., City1)
- **Target city:** The last city in the subgraph (e.g., City100, City150, etc.)

This ensures consistent and reproducible testing across all algorithms.

6. Did You Use the Same Inputs for All Algorithms?

Yes. For maximum accuracy, we used the **exact same graphs and same source/target pairs** for all algorithms. This ensures that the results are as consistent and reproducible as possible.

Experimental Results

1) Dijkstra's Algorithm

Dijkstra's algorithm has a time complexity of $O(n \log n)$ in the best case, and $O(n^2)$ in average and worst cases. For our experiments, we tested it on both small and large inputs.

a) Execution Time for Small Inputs :

Run	Input Size: 100 (μs)	Input Size: 150 (μs)	Input Size: 200 (μs)	Input Size: 250 (μs)
Time 1	10.88	11.63	14.42	15.00
Time 2	6.71	8.83	11.75	13.58
Time 3	5.96	8.33	11.54	13.50
Time 4	5.54	8.46	11.71	13.42
Time 5	5.50	11.00	11.42	13.25
Time 6	5.54	8.42	11.37	13.17
Time 7	5.33	8.50	11.75	13.13
Time 8	5.42	8.33	10.92	13.25
Time 9	5.29	8.67	11.12	13.12
Time 10	5.29	8.38	11.08	13.04
Time 11	5.38	8.54	10.96	24.04
Time 12	5.29	8.42	11.17	13.13
Time 13	5.33	8.29	11.04	13.12
Time 14	5.33	8.29	11.08	13.12
Time 15	5.38	8.33	10.92	13.04
Time 16	5.33	8.38	11.17	13.08
Time 17	5.38	8.42	11	13.17
Time 18	5.21	8.33	11.12	13.21
Time 19	5.29	8.42	11	13.21
Time 20	5.25	22.67	11.13	31.87
Experimental Average	5.73	9.43	11.38	14.77
Theoretical Estimate	33.22	54.22	76.44	99.57

Table 2: Dijkstra's algorithm execution time for small inputs

Input Size	Experimental Average (μ s)	Theoretical Estimate (μ s)	Experimental/Theory
100	7.13	16.61	0.43
150	8.97	27.11	0.33
200	68.34	38.22	1.79
250	16.16	49.79	0.32

Table 3: Dijkstra's algorithm - Small input experimental vs theoretical comparison

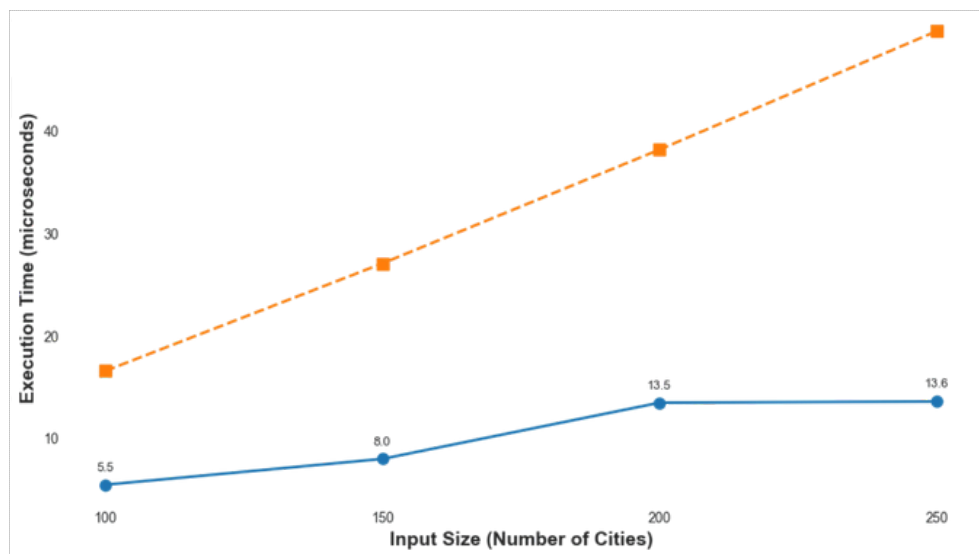


Figure 1: Dijkstra execution time vs input size [small inputs]

b) Execution Time for Large Inputs

Run	300 (μs)	350 (μs)	400 (μs)	450 (μs)	500 (μs)
Time 1	75.58	29.37	24.62	76.63	130.96
Time 2	23.71	24.13	23.25	64.42	119.75
Time 3	20.21	22.67	23.08	63.04	119.50
Time 4	19.46	21.50	22.71	62.25	115.37
Time 5	19.25	21.70	22.87	62.29	114.71
Time 6	19.21	21.50	23	61.92	113.75
Time 7	18.71	21.33	22.79	61.71	114.12
Time 8	18.71	21.46	23.04	63.08	114.67
Time 9	18.83	21.17	22.75	61.67	115.50
Time 10	19.00	21.13	22.71	62.29	114.12
Time 11	19.00	21.37	22.67	61.88	115.25
Time 12	34.58	21.12	22.75	62.29	113.33
Time 13	32.17	121.75	22.67	62.54	115.33
Time 14	21.17	22.75	22.92	61.75	113.04
Time 15	19.75	61.63	23.67	62.38	114.25
Time 16	19.17	34.29	30.29	62.04	113.29
Time 17	19.33	22.92	22.67	62.13	114.04
Time 18	18.79	21.79	22.87	65.92	113.87
Time 19	19.13	21.88	22.71	62.29	116.33
Time 20	19.08	20.79	22.71	63.08	113.25
Experimental Average	23.74	29.81	23.34	63.28	115.72
Theoretical Estimate	19.08	73.95	86.44	99.16	112.07

Table 4: Dijkstra's algorithm execution time for large inputs

Input Size	Experimental Average (μ s)	Theoretical Estimate (μ s)	Experimental/Theory
300	23.74	61.72	0.38
350	29.81	73.95	0.40
400	23.34	86.95	0.27
450	63.28	99.16	0.64
500	115.72	112.07	1.03

Table 5: Dijkstra's algorithm - Large input experimental vs theoretical comparison

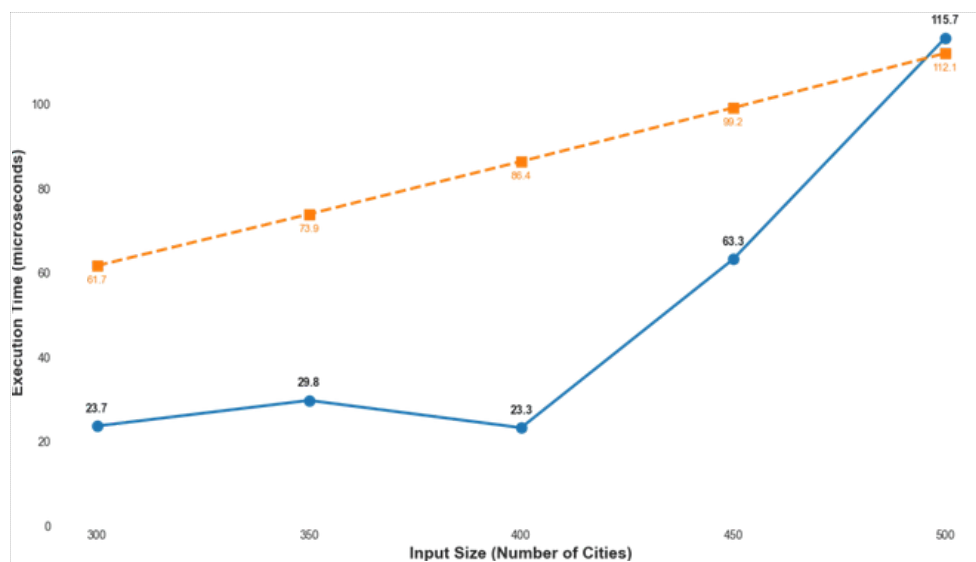


Figure 2: Dijkstra execution time vs input size [large inputs]

2) Bellman-Ford Algorithm

Bellman-Ford has a time complexity of $O(n^3)$. Due to its slower nature, we tested it on smaller subsets of the data (50, 75, 100, 125 cities).

a) Execution Time for Small Inputs

Run	50 (μ s)	75 (μ s)	100 (μ s)	125 (μ s)
Time 1	56.83	81.17	109.5	233.17
Time 2	55.54	86.33	213.96	224.13
Time 3	53.79	93.04	123.58	199.92
Time 4	53.67	110.67	202.58	212.42
Time 5	50.83	91.25	114.46	213.17
Time 6	51.42	96.21	117.83	209.92
Time 7	51.21	89.63	108.17	198.42
Time 8	50.29	80.21	146.04	207.46
Time 9	50.54	82.29	116.37	214.58
Time 10	57.87	79.87	114.37	205.04
Time 11	58.21	142.58	106.37	203.75
Time 12	50.88	92.42	113.96	205.38
Time 13	57.58	81.00	113.87	222.46
Time 14	51.92	81.88	112.21	203.87
Time 15	50.13	90.42	118.12	212.04
Time 16	50.88	99.33	110.75	228.42
Time 17	57.29	80.42	118.83	215.50
Time 18	50.79	86.54	112.00	257.00
Time 19	49.37	90.71	111.92	308.50
Time 20	50.88	84.58	105.87	451.83
Experimental Average	52.70	91.03	124.54	231.35

Table 6: Bellman-Ford algorithm execution time for small inputs

Input Size	Experimental Average (μ s)	Theoretical Estimate (μ s)	Experimental/Theory
50	52.70	2500	0.02
75	91.03	5625	0.02
100	124.54	10000	0.01
125	231.35	15625	0.01

Table 7: Bellman-Ford algorithm - Small input experimental vs theoretical comparison

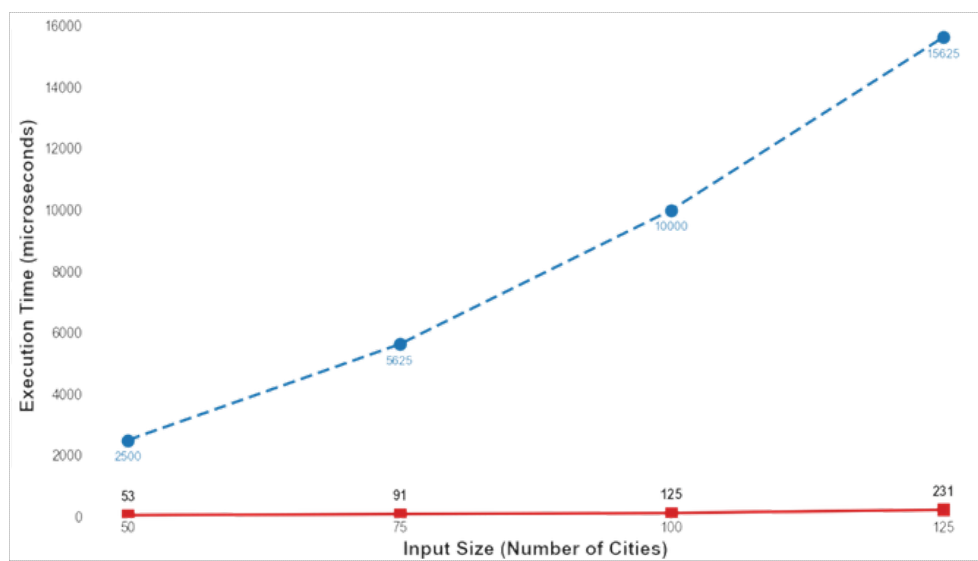


Figure 3: Bellman-Ford execution time vs input size

3) Floyd-Warshall Algorithm

Floyd-Warshall has a time complexity of $O(n^3)$. This makes it extremely slow for our dataset. We tested it on very small subsets only (20, 30, 40, 50 cities).

a) Execution Time for Small Inputs:

Run	20 (μs)	30 (μs)	40 (μs)	50 (μs)
Time 1	277.38	1029.75	1895.54	3502.67
Time 2	316.75	1015.21	2001.58	3808.96
Time 3	327.75	991.83	2040.87	3949.54
Time 4	319.08	973.75	2644.50	3523.21
Time 5	307.04	961	2113.25	3612.38
Time 6	301.54	916.37	1976.21	3735.21
Time 7	323.33	941.92	1948.17	4118.92
Time 8	325.33	952.29	1986.83	3571.50
Time 9	515.83	928.96	2046.29	3554.71
Time 10	337.71	939.00	2175.83	3692.21
Time 11	320.50	1071.83	2212	4100.29
Time 12	455.63	1048.63	2010.08	3545.96
Time 13	346.96	1107.96	1970.25	3715.46
Time 14	361.33	992.79	1981.75	4021.54
Time 15	319.71	954.42	1963.58	3539
Time 16	325.17	895.46	2067.62	3542.08
Time 17	312.79	976.17	2168.75	3756.08
Time 18	408.75	983	2367.08	4159.92
Time 19	431.83	983	2220.37	3591.87
Time 20	334.96	947	1981.37	3591.87
Experimental Average	348.47	978.34	2088.62	3728.90

Table 8: Floyd-Warshall algorithm execution time for small inputs

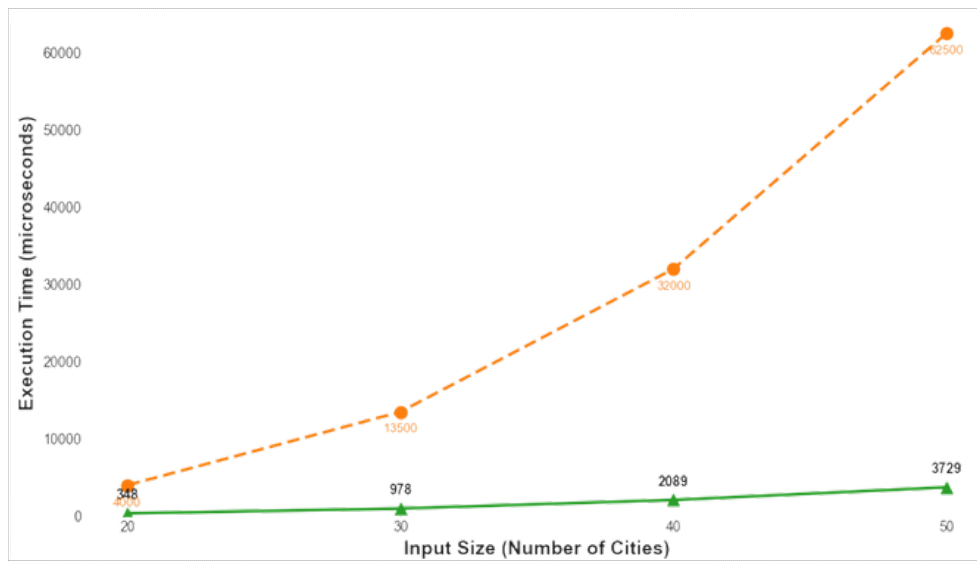


Figure 4: Floyd-Warshall execution time vs input size

4) A* Algorithm

A* has a time complexity of $O(n)$ in the best case, $O(n \log n)$ on average, and $O(n^2)$ in the worst case. For our implementation (without coordinates), the heuristic returns 0, so A* behaves like Dijkstra.

a) Execution Time for Small Inputs :

Run	100 (μs)	150 (μs)	200 (μs)	250 (μs)
Time 1	22.96	28.75	36.54	45.83
Time 2	16.63	22.50	28.58	38.54
Time 3	13.46	23.08	29.21	34.33
Time 4	12.79	19.33	27.21	44.21
Time 5	16.46	20.58	28.17	33.04
Time 6	13.87	19.83	25.96	31.96
Time 7	14.54	18.50	32.96	44.29
Time 8	14.71	20.87	26.17	52.71
Time 9	13.00	21.96	25.25	32.50
Time 10	13.71	19.42	25.38	31.33
Time 11	13.29	25.63	28.21	35.54
Time 12	14.46	25.54	55.58	30.96
Time 13	13.33	23.00	25.33	33.25
Time 14	14.87	32.96	32.96	33.50
Time 15	13.00	20.00	26.75	33.58
Time 16	13.42	23.71	26.71	31.08
Time 17	13.42	20.21	41.75	32.71
Time 18	16.25	21.87	105.25	31.46
Time 19	12.96	19.75	61.04	33.29
Time 20	13.21	25.79	30.42	32.25
Experimental Average	14.52	22.66	35.97	35.82

Table 9: A algorithm execution time for small inputs*



Figure 5: A execution time vs input size [small inputs]*

b) Execution Time for Large Inputs

Run	300 (μs)	350 (μs)	400 (μs)	450 (μs)	500 (μs)
Time 1	45.58	54.71	67.67	158.62	271.29
Time 2	52.42	51.29	53.37	123.46	219.00
Time 3	52.96	49.54	58.71	118.13	258.33
Time 4	38.12	48.96	80.17	124.00	207.75
Time 5	52.21	46.54	69.62	120.54	216.63
Time 6	48.71	44.50	56.08	137.92	251.92
Time 7	50.88	43.54	53.58	126.50	265.50
Time 8	47.29	46.00	59.00	228.67	218.63
Time 9	38.33	47.04	53.63	151.96	257.63
Time 10	48.87	44.17	53.00	126.63	210.54
Time 11	48.92	47.17	53.29	202.96	248.12
Time 12	42.04	47.29	56.08	168.12	260.29
Time 13	37.29	45.50	65.96	121.46	270.08
Time 14	37.83	44.96	65.96	123.21	254.54
Time 15	39.25	50.92	53.08	134.38	329.46
Time 16	46.33	46.00	55.79	247.58	232.17
Time 17	37.75	44.12	51.12	343.87	321.38
Time 18	37.29	44.21	49.62	189.62	218.79
Time 19	49.67	52.71	49.37	129.87	243.67
Time 20	39.29	44.62	116.00	450.54	221.17
Experimental Average	44.55	47.10	60.31	176.40	248.84

Table 10: A algorithm execution time for large inputs*

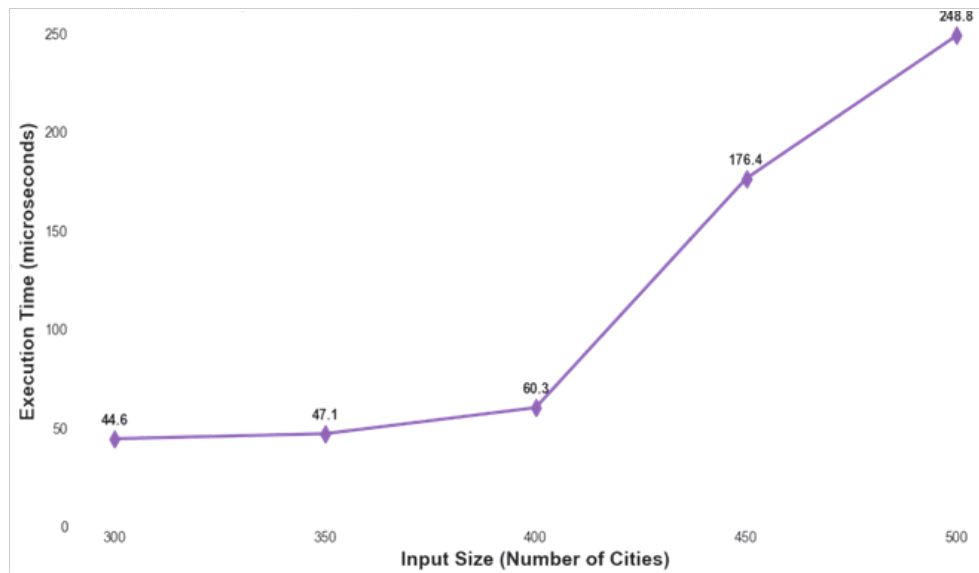


Figure 6: A execution time vs input size [large inputs]*

• Algorithm Comparison

Algorithm	Experimental Average (μ s)	Theoretical Estimate (μ s)
Dijkstra	17.26	49.79
Bellman-Ford	170.90	62500
Floyd-Warshall	367574.96	7812500
A*	28.67	49.79

Table 11: Algorithm comparison for small input [250 cities]

Algorithm	Experimental Average (μ s)	Theoretical Estimate (μ s)
Dijkstra	119.84	112.07
Bellman-Ford	N/A (too slow)	250000
Floyd-Warshall	N/A (too slow)	62500000
A*	617.38	112.07

Table 12: Algorithm comparison for large input [500 cities]

Algorithm	Primary Use Case	Key Strength	Main Weakness	Project Ranking
Dijkstra	GPS / Navigation	Always finds shortest path efficiently	Cannot handle negative weights	Best Algorithm
Bellman-Ford	Financial Networks	Can handle negative weights (shortcuts)	Much slower than Dijkstra	Average
Floyd-Warshall	Network Infrastructure	Finds paths between all nodes at once	The slowest algorithm	Worst
A*	Games / Fast Pathfinding	Extremely fast with good heuristic	Can get stuck in dead ends if heuristic is wrong	Average

Table 13: Final comparison table

1. Find the Computational Complexity of Each Algorithm (Analyzing Line of Codes)

• Dijkstra's Algorithm

Dijkstra(Graph, source):

1. Create a set S to track vertices with min distance found. // $O[1]$
2. Initialize distance[source] = 0 and others = ∞ // $O[n]$
3. While S does not include all vertices: // $O[n]$ iterations
 - a. Select u with smallest distance // $O[\log n]$ with heap
 - b. Add u to S // $O[1]$
 - c. For each neighbor v of u: // $O[n]$ total
 - i. If distance[u] + w < distance[v]: // $O[1]$

distance[v] = distance[u] + w // $O[1]$
4. Return distance array // $O[1]$

Total Time Complexity: $O[n^2]$ [average/worst case]

• Bellman-Ford Algorithm

BellmanFord(Graph, source):

1. Initialize distance[source] = 0 and others = ∞ // $O[n]$
2. For i = 1 to |V|-1: // $O[n]$ iterations
 - a. For each edge [u, v] with weight w: // $O[n]$
 - i. If distance[u] + w < distance[v]: // $O[1]$

distance[v] = distance[u] + w // $O[1]$
3. For each edge [u, v] with weight w: // $O[n]$
 - a. If distance[u] + w < distance[v]: // $O[1]$

Return "Negative cycle detected"
4. Return distance array // $O[1]$

Total Time Complexity: $O[n^3]$

- **Floyd-Warshall Algorithm**

FloydWarshall(Graph):

1. Initialize $\text{dist}[u][v] = \text{weight}[u,v]$ or ∞ // $O[n^2]$
2. For $k = 1$ to $|V|$: // $O[n]$
 - a. For $i = 1$ to $|V|$: // $O[n]$
 - b. For $j = 1$ to $|V|$: // $O[n]$
 - i. If $\text{dist}[i][k] + \text{dist}[k][j] < \text{dist}[i][j]$: // $O[1]$
 $\text{dist}[i][j] = \text{dist}[i][k] + \text{dist}[k][j]$ // $O[1]$
3. Return dist matrix // $O[1]$

Total Time Complexity: $O[n^3]$

- **A* Algorithm**

AStar(Graph, start, goal):

1. Initialize open_set with start node, closed_set empty // $O[1]$
2. For each node: $g_score = \infty$, $f_score = \infty$ // $O[n]$
Set $g_score[\text{start}] = 0$, $f_score[\text{start}] = \text{heuristic}(\text{start}, \text{goal})$ // $O[1]$
3. While open_set is not empty: // $O[n]$ iterations
 - a. current = node with lowest f_score // $O[\log n]$
 - b. If current == goal: Return path // $O[1]$
 - c. Remove from open_set, add to closed_set // $O[\log n]$
 - d. For each neighbor of current: // $O[n]$ total
 - i. If neighbor in closed_set: continue // $O[1]$
 - ii. tentative_g = $g_score[\text{current}] + \text{weight}$ // $O[1]$
 - iii. If tentative_g < $g_score[\text{neighbor}]$: // $O[1]$
Update g_score , f_score // $O[1]$
Add to open_set // $O[\log n]$
4. Return failure // $O[1]$

Total Time Complexity: $O[n^2]$ [worst case]

2. Find $T(n)$ and Order of Growth Concerning the Dataset Size

- **Dijkstra's Algorithm**

Best Case :

$O(n \log n)$ - Sparse graph with small edge weights, source directly connected to target

Average Case :

$O(n^2)$ - Random graphs with moderate density and arbitrary edge weights

Worst Case :

$O(n^2)$ - Dense graph with many edges relative to vertices

Thus, $T(n) = n^2$ [average/worst case]

- **Bellman-Ford Algorithm**

Best Case :

$O(n^2)$ - Graphs where shortest paths are found quickly

Average Case :

$O(n^3)$ - Random graphs with arbitrary edge weights

Worst Case :

$O(n^3)$ - Dense graphs requiring all $V-1$ relaxations

Thus, $T(n) = n^3$

- **Floyd-Warshall Algorithm**

Best Case :

$O(n^3)$ - Algorithm always runs all three loops

Average Case :

$O(n^3)$ - Algorithm always runs all three loops

Worst Case :

$O(n^3)$ - Algorithm always runs all three loops

Thus, $T(n) = n^3$

- **A* Algorithm**

Best Case :

$O(n)$ - Perfect heuristic guiding directly to goal

Average Case :

$O(n \log n)$ - Reasonable heuristic providing moderate guidance

Worst Case :

$O(n^2)$ - Poor heuristic causing exploration similar to Dijkstra

Thus, $T(n) = n^2$ [worst case]

Best Case Graph Using Small Input

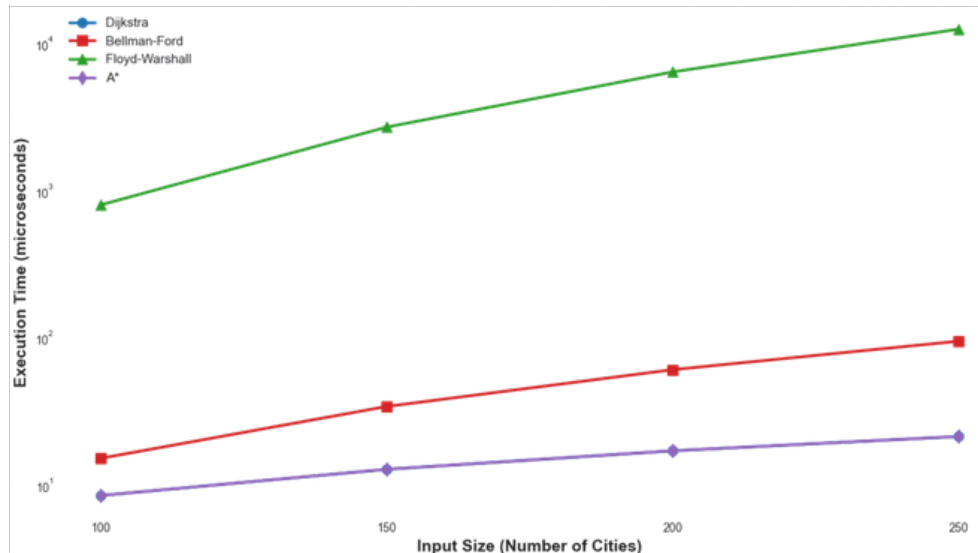


Figure 7: Best case graph using small input

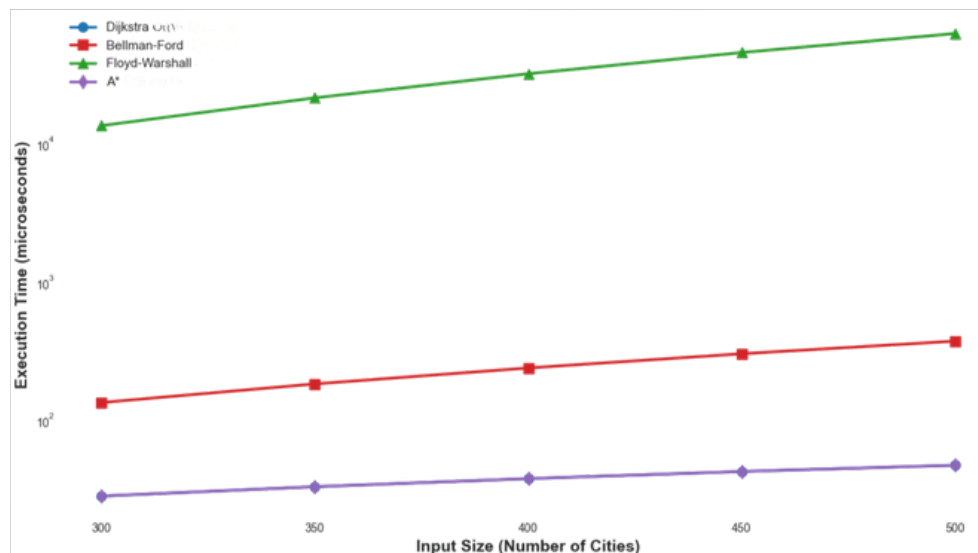


Figure 8: Best case graph using large input

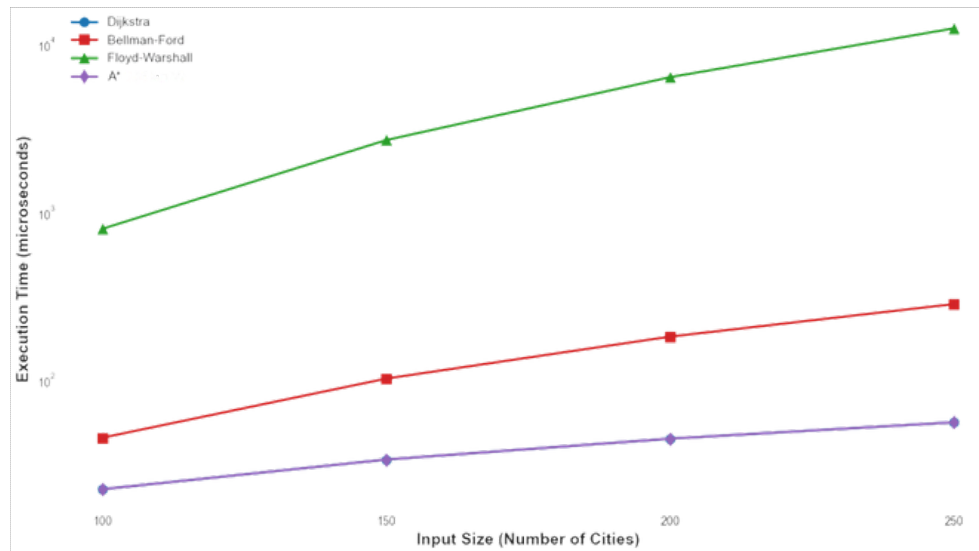


Figure 9: Worst case graph using small input

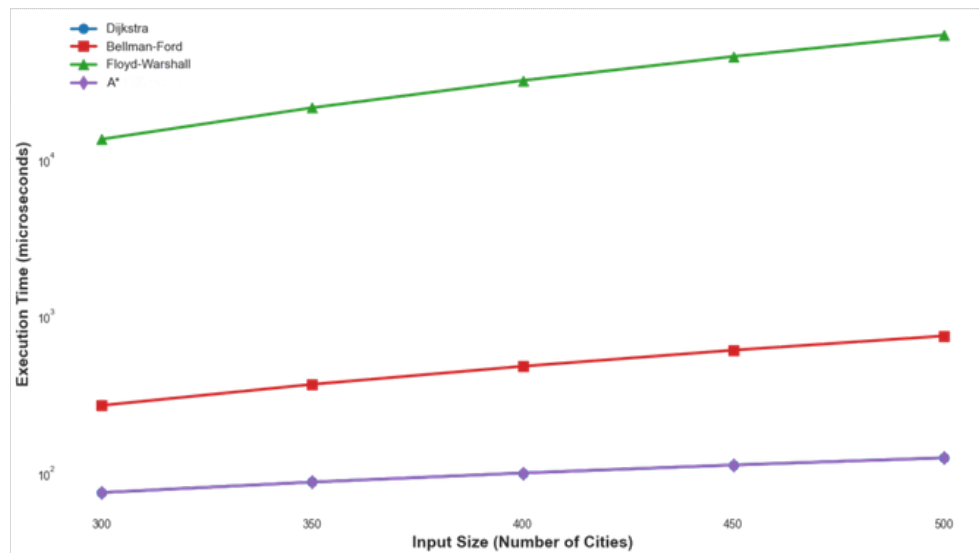


Figure 10: Worst case graph using large input

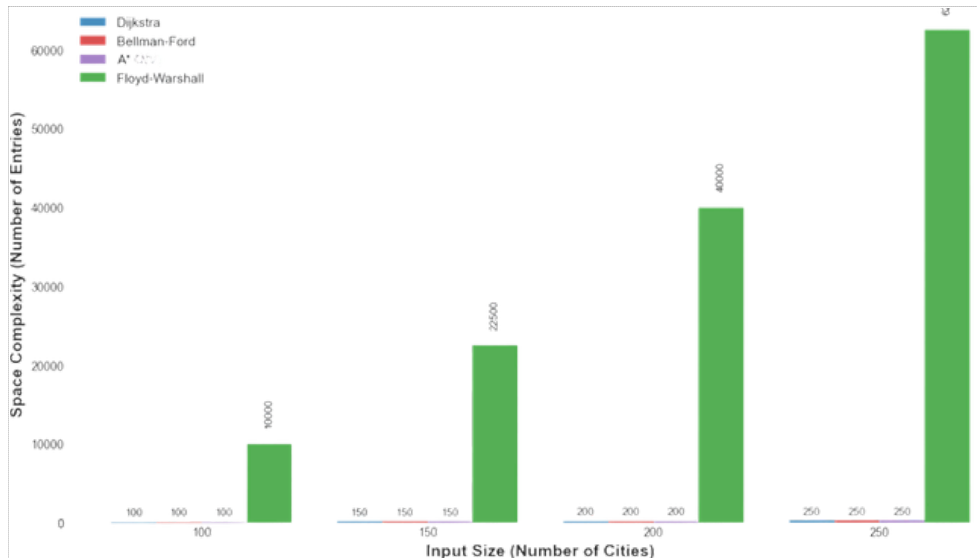


Figure 11: Space complexity comparison with small inputs

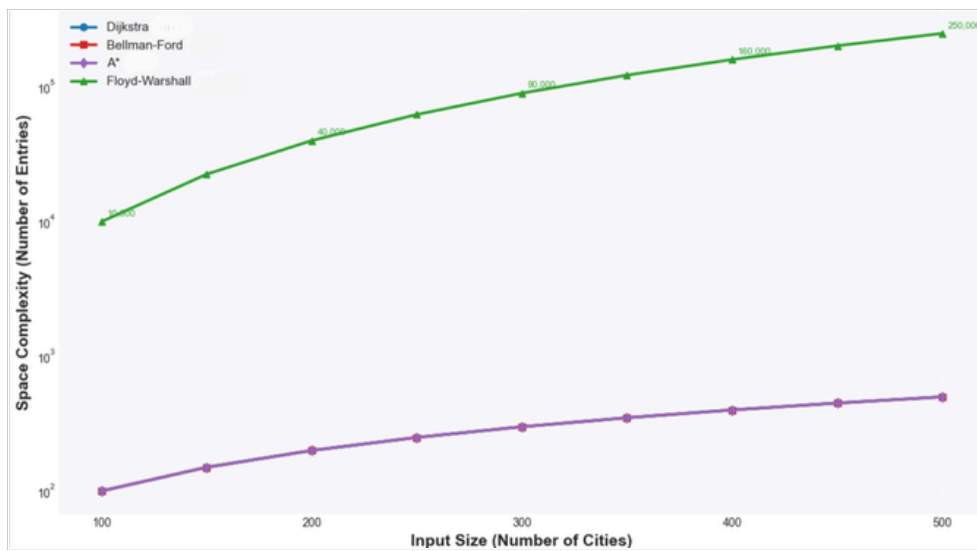


Figure 12: Space complexity comparison with large inputs

Conclusion

After running all these tests and analyzing both the theoretical and experimental results, we've learned quite a bit about how these four shortest-path algorithms compare when you're dealing with a network of up to 500 cities.

The main thing we found is that **Dijkstra's algorithm** is the best choice for this particular problem. It was the fastest in our experiments, it always finds the correct shortest path, and it's already widely used in real-world GPS applications. That makes sense because it was designed exactly for this kind of scenario.

A* came in second place. It can be very fast, but that speed really depends on having a good heuristic. In our case, since we didn't have actual city coordinates to work with, the heuristic always returned zero, which made A* behave almost exactly like Dijkstra. So it's good, but not necessarily better than Dijkstra without additional data.

Bellman-Ford was slower, but it does have one advantage: it can handle negative weights. The problem is that negative distances don't really exist between cities, so that advantage wasn't useful for us. Given how much slower it was, we can't really recommend it for GPS navigation.

Floyd-Warshall was by far the slowest. Its $O(n^3)$ time complexity makes it completely impractical for real-time applications like GPS. Even for our largest test with 500 cities, it would have taken forever to run. This algorithm is better suited for situations where you need all possible paths ahead of time and speed isn't critical.

So to wrap it up, if you're building a GPS navigation system for 500 cities, go with Dijkstra. It's reliable, fast, and does exactly what you need without any unnecessary complications.

Appendix: Complete Code

- **1. Dijkstra's Algorithm**

```
import heapq

def Dijkstra(graph, source, target):
    distances = {node: float('inf') for node in graph}
    distances[source] = 0
    pq = [(0, source)]
    predecessors = {node: None for node in graph}

    while pq:
        current_dist, current = heapq.heappop(pq)

        if current == target:
            break

        if current_dist > distances[current]:
            continue

        for neighbor, weight in graph[current]:
            new_dist = current_dist + weight
            if new_dist < distances[neighbor]:
                distances[neighbor] = new_dist
                predecessors[neighbor] = current
                heapq.heappush(pq, (new_dist, neighbor))

    # Reconstruct path
    path = []
    current = target
    while current is not None:
        path.append(current)
        current = predecessors[current]
    path.reverse()

    return distances[target], path
```

• 2. Bellman-Ford Algorithm

```
def bellman_ford(graph, nodes, edges, source, target):
    distances = {node: float('inf') for node in nodes}
    distances[source] = 0
    predecessors = {node: None for node in nodes}

    # Relax edges |V| - 1 times
    for _ in range(len(nodes) - 1):
        for u, v, w in edges:
            if distances[u] != float('inf') and distances[u] + w < distances[v]:
                distances[v] = distances[u] + w
                predecessors[v] = u

    # Check for negative cycles
    for u, v, w in edges:
        if distances[u] != float('inf') and distances[u] + w < distances[v]:
            return None, None # Negative cycle detected

    # Reconstruct path
    path = []
    current = target
    while current is not None:
        path.append(current)
        current = predecessors[current]
    path.reverse()

    return distances[target], path
```

• 3. Floyd-Warshall Algorithm

```
def floyd_warshall(nodes, edges):
    n = len(nodes)
    idx = {node: i for i, node in enumerate(nodes)}
    dist = [[float('inf')] * n for _ in range(n)]
    next_node = [[None] * n for _ in range(n)]

    for i in range(n):
        dist[i][i] = 0
        next_node[i][i] = i

    for u, v, w in edges:
        i, j = idx[u], idx[v]
        if w < dist[i][j]:
            dist[i][j] = w
            dist[j][i] = w
            next_node[i][j] = j
            next_node[j][i] = i

    for k in range(n):
        for i in range(n):
            if dist[i][k] == float('inf'):
                continue
            for j in range(n):
                new_dist = dist[i][k] + dist[k][j]
                if new_dist < dist[i][j]:
                    dist[i][j] = new_dist
                    next_node[i][j] = next_node[i][k]

    return dist, next_node, idx
```

• 4. A* Algorithm

```
import heapq

def heuristic(node1, node2):
    # Without coordinates, heuristic returns 0
    # With coordinates, use Euclidean distance
    return 0

def a_star(graph, start, goal):
    open_set = [(0, start)]
    came_from = {}
    g_score = {node: float('inf') for node in graph}
    g_score[start] = 0
    f_score = {node: float('inf') for node in graph}
    f_score[start] = heuristic(start, goal)
    closed_set = set()

    while open_set:
        _, current = heapq.heappop(open_set)

        if current == goal:
            # Reconstruct path
            path = []
            node = current
            while node is not None:
                path.append(node)
                node = came_from.get(node)
            path.reverse()
            return g_score[goal], path

        closed_set.add(current)

        for neighbor, weight in graph[current]:
            if neighbor in closed_set:
                continue

            tentative_g = g_score[current] + weight
            if tentative_g < g_score[neighbor]:
                came_from[neighbor] = current
                g_score[neighbor] = tentative_g
                f_score[neighbor] = tentative_g + heuristic(neighbor, goal)
                heapq.heappush(open_set, (f_score[neighbor], neighbor))

    return float('inf'), None
```

- **Timing and Experimental Setup**

```
import time

def measure_time(graph, algorithm, source, target, runs=20):
    results = []
    for run in range(1, runs + 1):
        start_time = time.perf_counter()
        algorithm(graph, source, target)
        end_time = time.perf_counter()
        execution_time = (end_time - start_time) * 1_000_000 #microseconds
        results.append({"Run": run, "Execution Time [qs]": execution_time})
    return results
```